

Selenium WebDriver ?

Selenium WebDriver is an open-source browser automation library that lets you programmatically control a browser — clicking, typing, navigating — exactly as a real user would. It communicates with browsers through their native automation APIs.

- **WebDriver** — the core API your Java code calls to drive a browser
- **Selenium Grid** — runs the same tests across many browsers/machines in parallel
- **Selenium IDE** — record-and-playback browser extension (not used in professional frameworks)

Note : WebDriver talks to the browser through a driver executable (ChromeDriver, GeckoDriver). Your **Java code** → **WebDriver API** → **driver exe** → **browser**.

Important Dependencies

- Selenium-java
- webdrivermanager

```
←!— Selenium Java bindings →  
<dependency>  
  <groupId>org.seleniumhq.selenium</groupId>  
  <artifactId>selenium-java</artifactId>  
  <version>4.18.1</version>  
</dependency>  
  
←!— WebDriverManager: auto-downloads matching driver →  
<dependency>  
  <groupId>io.github.bonigarcia</groupId>  
  <artifactId>webdrivermanager</artifactId>  
  <version>5.7.0</version>  
</dependency>
```

WebDriverManager eliminates the need to manually download ChromeDriver or GeckoDriver. Call `WebDriverManager.chromedriver().setup()` before instantiating the driver and it auto-matches the driver version to your installed browser.

Project Structure

```
DIRECTORY LAYOUT

my-selenium-project/
├─ pom.xml           # dependency + build config
└─ src/
    ├─ main/java/    # page objects, utilities
    │   └─ pages/
    │       └─ LoginPage.java
    └─ test/java/    # test classes go here
        └─ tests/
            └─ LoginTest.java
```

Browser Drivers

DRIVER	BROWSER	WHEN TO USE
ChromeDriver	Google Chrome	Default choice – Chrome has the largest market share, fastest DevTools support, best compatibility with Selenium 4
GeckoDriver	Mozilla Firefox	Use for cross-browser coverage. Firefox handles some edge cases (e.g. strict cookie policies) that Chrome doesn't surface
EdgeDriver	Microsoft Edge	Chromium-based, behaves similarly to Chrome. Required for enterprise environments where Edge is the corporate browser

INSTANTIATING DRIVERS WITH WEBDRIVERMANAGER

// Chrome

`WebDriverManager.chromedriver().setup();`

`WebDriver driver = new ChromeDriver();`

// Firefox

`WebDriverManager.firefoxdriver().setup();`

`WebDriver driver = new FirefoxDriver();`

// Edge

`WebDriverManager.edgedriver().setup();`

`WebDriver driver = new EdgeDriver();`

Browser Automation & Locators

After getting the **WebDriver** instance then we have core methods for controlling the browser life cycle and navigation.

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import io.github.bonigarcia.wdm.WebDriverManager;

public class FirstTest {
    public static void main(String[] args) {

        WebDriverManager.chromedriver().setup();
        WebDriver driver = new ChromeDriver();

        // Open a URL
        driver.get("https://example.com");

        // Alternative navigation methods
        driver.navigate().to("https://example.com");
        driver.navigate().back();           // browser back button
        driver.navigate().forward();        // browser forward
        button
        driver.navigate().refresh();         // refresh page

        // Window management
        driver.manage().window().maximize();

        // Get page info
        String title = driver.getTitle();
        String url   = driver.getCurrentUrl();

        // Always close the browser at the end
        driver.quit(); // closes all windows + kills driver
        process
    }
}
```

METHOD	WHAT IT DOES
<code>driver.get(url)</code>	Opens URL and waits for page load event
<code>navigate().to(url)</code>	Same as <code>get()</code> but part of the Navigation API (supports back/forward)
<code>driver.close()</code>	Closes current tab only
<code>driver.quit()</code>	Closes all windows AND kills the driver process – always prefer this

Always call `driver.quit()` (not `close()`) at the end of a test. `close()` only closes the current tab — the driver process keeps running, leaking memory.

Element Locator Strategies

All locators are passed to `driver.findElement(By.xxx())`. Choose the most stable strategy for each element.

LOCATOR	EXAMPLE	STABILITY	NOTES
<code>By.id()</code>	<code>By.id("username")</code>	★★★★★	Always first choice if id exists and is unique
<code>By.name()</code>	<code>By.name("email")</code>	★★★★	Common in forms; can be non-unique on a page
<code>By.className()</code>	<code>By.className("btn-primary")</code>	★★★	Matches first element with that class; risky if class is reused
<code>By.tagName()</code>	<code>By.tagName("h1")</code>	★★	Returns first matching tag; use only when element is unique by tag
<code>By.linkText()</code>	<code>By.linkText("Sign In")</code>	★★★★	Exact match of anchor text; breaks if wording changes
<code>By.partialLinkText()</code>	<code>By.partialLinkText("Sign")</code>	★★★	Partial match; useful for truncated or long link text
<code>By.cssSelector()</code>	<code>By.cssSelector("#login .btn")</code>	★★★★★	Faster than XPath; expressive but no upward traversal
<code>By.xpath()</code>	<code>By.xpath("//input[@id='user']")</code>	★★★★★	Most flexible; can traverse parent nodes; slower than CSS

LOCATOR	EXAMPLE	STABILITY	NOTES
<code>By.id()</code>	<code>By.id("username")</code>	★★★★★	Always first choice if id exists and is unique
<code>By.name()</code>	<code>By.name("email")</code>	★★★★	Common in forms; can be non-unique on a page
<code>By.className()</code>	<code>By.className("btn-primary")</code>	★★★	Matches first element with that class; risky if class is reused
<code>By.tagName()</code>	<code>By.tagName("h1")</code>	★★	Returns first matching tag; use only when element is unique by tag
<code>By.linkText()</code>	<code>By.linkText("Sign In")</code>	★★★★★	Exact match of anchor text; breaks if wording changes
<code>By.partialLinkText()</code>	<code>By.partialLinkText("Sign")</code>	★★★	Partial match; useful for truncated or long link text
<code>By.cssSelector()</code>	<code>By.cssSelector("#login .btn")</code>	★★★★★	Faster than XPath; expressive but no upward traversal
<code>By.xpath()</code>	<code>By.xpath("//input[@id='user']")</code>	★★★★★	Most flexible; can traverse parent nodes; slower than CSS

```

FINDING ELEMENTS — SYNTAX REFERENCE
WebElement el;

// By ID — most stable
el = driver.findElement(By.id("username"));

// By CSS selector
el = driver.findElement(By.cssSelector("input[name='email']"));
el = driver.findElement(By.cssSelector("#loginForm .submit-btn"));

// By XPath
el = driver.findElement(By.xpath("//button[text()='Login']"));
el = driver.findElement(By.xpath("//input[@placeholder='Enter email']"));

// Find multiple elements
List<WebElement> links = driver.findElements(By.tagName("a"));

```

XPath vs CSS Selectors — when to use which

CSS selectors are faster — the browser's native CSS engine evaluates them directly. XPath requires an additional XML parsing step. In large DOMs, this difference is noticeable across thousands of test steps.

XPath is more flexible — it can traverse parent nodes (CSS cannot move "upward" in the DOM), match by text content, and use complex conditions with and/or.

SCENARIO	USE
Element has a unique id or class	CSS — <code>#id</code> or <code>.class</code>
Match by visible text inside element	XPath — <code>//button[text()='Submit']</code>
Navigate to parent element	XPath — <code>//span[@class='err']/..</code>
Attribute contains partial value	CSS — <code>input[id*='user']</code> or XPath <code>contains(@id,'user')</code>
nth child or sibling	CSS — <code>tr:nth-child(3) td</code>
Combine multiple attribute conditions	XPath — <code>//input[@type='text' and @name='q']</code>

KEY XPATH TECHNIQUES

```
// Absolute XPath (fragile - avoid)
"/html/body/div[2]/form/input[1]"

// Relative XPath (preferred)
"//input[@id='username']"

// contains() - handles partial attribute values
"//input[contains(@id,'user')]"

// text() - match by visible text
"//button[text()='Submit']"

// normalize-space() - handles whitespace
"//button[normalize-space()='Sign In']"

// Navigate to parent (find label of a broken field)
"//input[@id='email']/.."

// Sibling - following-sibling
"//label[text()='Email']/following-sibling::input"
```

KEY CSS SELECTOR TECHNIQUES

```
/* Attribute exact match */


/* Attribute contains */


/* Attribute starts with */


/* Attribute ends with */


/* Direct child */
form > input

/* Descendant */
#loginForm input.email-field

/* nth-child */
table tr:nth-child(2) td
```


Dynamic locators for changing UIs

Many modern apps generate element IDs dynamically (e.g. input_78324). These locators break every deployment.

Use

- Use contains() or starts-with() in XPath to match stable parts of changing attributes
- Anchor to stable parent elements and navigate down relatively
- Use visible text, placeholder, aria-label, or data-testid attributes (added by devs for automation)
- Avoid using absolute XPath — it breaks with any DOM structure change

```
DYNAMIC ELEMENT STRATEGIES
// BAD - dynamic ID, breaks every build
driver.findElement(By.id("input_78324"));

// GOOD - contains() on stable partial id
driver.findElement(By.xpath("//input[contains(@id,'email')]"));

// GOOD - anchor by label text, then target sibling input
driver.findElement(By.xpath(
    "//label[text()='Email address']/following-sibling::input"));

// GOOD - use placeholder or aria-label
driver.findElement(By.cssSelector("input[placeholder='Enter your email']"));

// BEST - data-testid attribute added for automation
driver.findElement(By.cssSelector("[data-testid='email-input']"));
```

Note : Ask your dev team to add data-testid attributes to key elements. They don't affect styling or functionality, and make locators 100% immune to DOM restructuring.

Phase 1 & 2 — Key takeaways

- Maven + WebDriverManager = zero manual driver management. Set it up once.
- Prefer By.id() → CSS selector → XPath, in that order of preference
- Never use absolute XPath. Always use relative XPath starting with //
- CSS is faster; XPath is more powerful. Use each where it fits best
- Use data-testid attributes for elements that lack stable natural identifiers
- Always driver.quit() in an @AfterMethod — never leave driver processes running
- driver.get() and navigate().to() both load URLs; navigate() additionally gives you back/forward/refresh